PROBLEM STATEMENT 12 · AI FOR AEROSPACE & MECHANICAL ENGINEERING

VISION-PHM

Ten solution paradigms, benchmarked against industry practice, distilled into one decision-first, uncertainty-calibrated architecture for aero-engine Remaining Useful Life

−2.3 / −3.2
cycles of RMSE removed by per-regime normalisation (FD002 / FD004), paired 95% CI excludes zero

5–8×
less unused engine life than fixed-interval maintenance at the same failure-risk budget (held-out engines)

42% → ≤4%
held-out splits breaking a 5% failure target, naive vs risk-controlled thresholds (7 of 8 cases)

±2.3
cycles: half-width of one model's 95% RMSE interval on FD001. A 1-cycle leaderboard gap is noise

All four figures are **measured on the real NASA C-MAPSS files** in this work (code supplied).
Prepared 30 October 2026 · Evidence legend: MEASURED / LITERATURE / DESIGN used throughout.
The headline numbers come from a gradient-boosting proxy; no neural network was trained (see Section 7.0).

WHAT IS INSIDE

Ten solutions	Ten different paradigms (deep sequence, health-index analogs, physics filters, foundation models, Bayesian, survival, hybrid twin, anomaly, federated, RL), each with its fatal flaw and what to keep
Industry benchmark	What GE Aerospace, Rolls-Royce, Pratt & Whitney, CFM/Safran and Airbus Skywise publicly document, eight gaps in current practice, and a 14-row flaw → improvement → innovation ledger
Final architecture	Seven layers plus a trust envelope and fleet-learning loop; the mathematics of calibration, optimal stopping and risk-controlled triggers
Real-data evidence	Four experiments on NASA C-MAPSS, including the results that did not favour the design
Execution	48-hour team plan, scope tiers, demo script, judge Q&A, risk register, tested reference code

Contents

0. Executive summary
1. Reframing the problem
   - 1.1 What the brief really asks for
   - 1.2 Six traps hidden in the problem statement
   - 1.3 Metric hierarchy used in this document
2. Ground truth: what the best organisations actually run
   - 2.1 System-by-system
   - 2.2 Five patterns common to all of them
   - 2.3 Eight gaps in current practice
3. Ten completely different solutions
4. Head-to-head comparison
   - 4.1 Scored matrix
   - 4.2 Capability matrix against industry
   - 4.3 What each solution would have to beat
5. Flaw → improvement → innovation ledger
6. The final solution: VISION-PHM
   - 6.1 Reference architecture
   - 6.2 Design principles
   - 6.3 Layer specification
   - 6.4 The mathematical core
   - 6.5 Leakage-safe data and training protocol
   - 6.6 Deployment topology and scale
7. Evidence from real C-MAPSS data
   - 7.0 What was run, and what was not
   - 7.1 Protocol effect and statistical noise
   - 7.2 Calibration: helpful, but not automatic
   - 7.3 Decision value: prognosis versus calendar
   - 7.4 Held-out threshold selection: the finding that changed the design
   - 7.5 What these results do and do not establish
8. Evaluation scorecard
9. Hackathon execution plan
   - 9.1 Roles and timeline (48 hours, four people)
   - 9.2 Scope tiers
   - 9.3 Five-minute demo script
   - 9.4 Questions to be ready for
   - 9.5 Repository layout
   - 9.6 Risk register
10. Limitations and the path to a real fleet

Appendix A. Reference code

Appendix B. C-MAPSS cheat-sheet

Appendix C. Reproducing Section 7

References

0. Executive summary

The brief asks for a Remaining-Useful-Life regressor. The industry does not run one. The public material from Rolls-Royce, GE Aerospace, Pratt & Whitney, CFM/Safran and Airbus describes *monitor → alert → engineer review → planned intervention* pipelines [2, 5, 9, 12, 13]: alerts, trend shifts, fleet dashboards and shop-visit forecasts. In the sources I reviewed, none publishes an independently checkable RUL error, a calibration curve or a failure-rate guarantee (they may exist internally). A prototype that wins on RMSE alone would be a better version of the wrong thing.

This document therefore does four things:

1. **Designs ten fundamentally different solutions** to Problem Statement 12 (Section 3) and scores them against nine criteria that matter in a real fleet, not just on a leaderboard.
2. **Benchmarks all ten against what the top-tier operators and OEMs actually run today** (Sections 2, 4) and ledgers every flaw, improvement and innovation (Section 5).
3. **Synthesises one final architecture, VISION-PHM** (Section 6): a seven-layer, decision-first, uncertainty-calibrated prognostics stack in which every component exists because an experiment, a source or a documented industry gap justified it.
4. **Tests the load-bearing claims on the real NASA C-MAPSS data** (Section 7) and reports the results honestly, including the ones that did not go the way the design hoped.

> **The one-sentence thesis.** *RUL is not the product, the maintenance decision is. So the system's job is to turn degradation telemetry into a calibrated risk statement and then into a schedule that provably respects a failure-risk budget at minimum wasted life.*

Five non-obvious findings this work is built on

#	Finding	Evidence
1	**The brief names sensors C-MAPSS does not contain.** The problem statement lists vibration and oil pressure; C-MAPSS has 21 gas-path/speed/flow channels and neither. The architecture must be modality-agnostic, and the demo must be honest about it.	Dataset documentation [20, 21]; Appendix B
2	**On a 100-engine test set, a 1-cycle RMSE gap is noise.** One model's 95% bootstrap interval on FD001 spans 11.4 to 16.1 cycles. Leaderboard-chasing on FD001/FD003 is statistically meaningless; decision-level metrics are not.	MEASURED Section 7.1
3	**Per-regime normalisation is a large, nearly free gain on FD002/FD004.** It cut RMSE by 2.3 and 3.2 cycles (paired bootstrap CIs exclude zero), a bigger effect than the one-to-two-cycle gaps that separate many published architectures.	MEASURED Section 7.1; consistent with [23]
4	**Any prognostic trigger beat fixed-interval maintenance by about 5–8× in unused life at the same failure risk (held-out)**, but a point-RUL threshold tuned on the same engines silently *breaks* its own risk target in up to 42% of held-out splits. Risk-controlled threshold selection fixes this in 7 of 8 policy/dataset cases.	MEASURED Sections 7.3–7.4
5	**Conformal calibration helps, but plain calibration under-covers** (83–88% observed at 90% nominal). The calibration set must mimic how the model is used in deployment. This is a design requirement, not a footnote.	MEASURED Section 7.2

Evidence legend used throughout

MEASURED run by me in this work on real C-MAPSS files (code supplied). LITERATURE taken from a cited source; not re-run. DESIGN proposed; not built or tested here.

1. Reframing the problem

1.1 What the brief really asks for

The statement has three verbs: *evaluate degradation trajectories*, *estimate RUL*, *schedule optimal maintenance*. Most submissions will deliver the second verb and gesture at the third. The scoring dimensions a serious panel will probe are:

Dimension	What a shallow answer shows	What a serious answer shows
End-to-end functionality	Notebook with a trained LSTM	Streaming replay → model → calibrated risk → schedule → dashboard
Scalability	"Can run on cloud"	Per-engine state is O(window); fleet cost estimated; edge vs. cloud split justified
Real-world applicability	Cites "Industry 4.0"	Handles censoring, regime changes, sensor faults, abstention, certification posture
Prognostic quality	One RMSE number	RMSE **with confidence intervals**, NASA score, interval coverage, decision KPIs

1.2 Six traps hidden in the problem statement

1. **Missing modalities.** C-MAPSS has no vibration or oil-pressure channel (Appendix B). The closest thing to exhaust-gas temperature is T50 (low-pressure-turbine outlet temperature); spool speeds are Nf and Nc. Claiming a vibration model trained on C-MAPSS would be fabrication. *Response:* a modality-adapter interface plus a channel-dropout robustness test, and an explicit pointer to N-CMAPSS [21] and

public bearing datasets for the vibration branch.

2. **It is simulated, run-to-failure data.** Real engines are removed before failure. Labels in a real fleet are *right-censored*. *Response:* a hazard head trained with a censored likelihood (Section 6).
3. **Tiny test sets.** FD001 and FD003 have 100 test engines each. Differences below roughly 2 cycles RMSE are not distinguishable (Finding 2).
4. **Evaluation protocol drift.** Published C-MAPSS numbers depend on normalisation scope, RUL cap, window choice, and whether the final model was refit on validation data [23]. *Response:* a fixed, leakage-safe protocol (Section 6.5).
5. **Asymmetric cost.** Predicting failure too late is worse than too early. The NASA scoring function encodes that asymmetry (late errors are penalised exponentially faster), yet most models train on symmetric MSE.
6. **RUL ≠ decision.** A number with no error bar cannot be scheduled against. Section 6.3 (L4/L5) closes that gap.

1.3 Metric hierarchy used in this document

Tier A, prognostic: test RMSE and NASA score per subset with bootstrap CIs. **Tier B, probabilistic:** empirical coverage of intervals (PICP) and mean interval width (MPIW). **Tier C, decision:** in-service failure rate versus mean unused life at removal; false-alarm rate; warning lead time. **Tier D, operational:** latency, memory, drift detection delay, abstention rate. A solution that only reports Tier A is incomplete.

2. Ground truth: what the best organisations actually run

All entries below summarise *public* sources reviewed for this document. A dash means "not found in the sources I reviewed", **not** "does not exist"; proprietary systems are by nature under-documented.

2.1 System-by-system

Organisation / system	What public sources document	Observed limits
GE Aerospace: per-engine digital twins, Fleet Monitor X, AI/ML 24×7 monitoring	A digital replica exists for each engine built at its Lafayette plant, fed by data downloaded per flight [1]. Fleet Monitor X focuses on data flow, engine-health monitoring, alert management and customer notification reports, aiming to cut unexpected engine removals [2]. An AI-based material assistant forecasts LEAP shop-visit workscope nine months ahead [3]; digital-twin models forecast workscope and parts months before induction [4].	No public accuracy, calibration or false-alarm statistics. Forecasting is strongest on the MRO/material side.
Rolls-Royce: Engine Health Monitoring, IntelligentEngine, Blue Data Thread	Standard EHM uses snapshot data (about five data points per flight); advanced EHM streams continuous data up to one point per second; the Pearl-family EVHMU exposes over 10,000 parameters, uploads high-frequency vibration after landing and supports two-way reconfiguration [5]. Customer data reaches Rolls-Royce through an automated pipeline [8]. Digital twins are described as predicting maintenance demand "weeks, days and even years" ahead [6]. A reported 48% increase in time to first removal is labelled *initial anecdotal* by the source [7].	Headline gains are vendor-reported and anecdotal. A goal of "zero false predictions" is stated as an aspiration [7], not a measured rate.
Pratt & Whitney: ADEM / EngineWise	ADEM analyses more than 10,000 engines across 140 customers [9]; full-flight data replaces take-off/cruise snapshots through the e-FAST ecosystem and a Teledyne data-delivery partnership [10, 11]. Offerings are wrapped in dollar-per-flight-hour fleet-management programmes [11].	"Expert analysis of real-time health data" is central [11]: the human engineer is the bottleneck.
CFM / Safran: LEAP health monitoring	Models multi-sensor data at take-off, climb and cruise using probabilistic diagnostic and prognostic ML, producing targeted alerts from known operating signatures; reports 60% earlier lead time for preventive-maintenance recommendations [12].	Alert-centric; "earlier lead time" has no published definition of baseline.
Airbus Skywise (Palantir-based platform): Reliability, Predictive Maintenance, Health Monitoring	Skywise Health Monitoring collates alerts, flight-deck effects and maintenance messages, prioritises them, links faults to troubleshooting procedures and shows maintenance history [13, 14].	Airframe/aircraft-level view; engine prognostics depend on OEM data access.
Lufthansa Technik AVIATAR, MTU myEFM/myETM	AVIATAR aims at a replica of every aircraft in an airline's fleet, feeding engine-health data through digital twins [15]. MTU's fleet-management and trend-monitoring platforms apply AI to engine maintenance planning [16].	Operator-side platforms; detail of the prognostic core not public.

2.2 Five patterns common to all of them

1. **Sense → downlink → ground analytics.** Data volume is rising: snapshots → full-flight → continuous streams [5, 11].
2. **Baseline-and-delta monitoring.** Deviation from a healthy, condition-corrected baseline (the classical *EGT margin* idea) is the backbone; monitoring exhaust-gas temperature is cited as the classical way to judge when an overhaul is due [1].
3. **ML for early detection and workscope forecasting**, usually probabilistic or signature-based [3, 12].
4. **Human experts close the loop.** Alerts go to engineers, then to customer reports and recommendations [5, 11, 13].
5. **A commercial wrapper**: service agreements and flat-rate fleet programmes, so prediction quality maps to the provider's own cost [11].

2.3 Eight gaps in current practice

*Evidence tags: **S** = supported by a cited source; **A** = my analysis, to be challenged.*

#	Gap	Tag	Why it matters
G1	**Alert-centric, not decision-centric.** Outputs are alerts and reports; optimal intervention timing is a separate human/ERP step.	S [13] / A	A calibrated risk number could be *scheduled against* directly.
G2	**No public, independent calibration evidence.** Gains are reported as percentages or anecdotes [7, 12].	S	Cannot audit false-negative risk, the metric safety cares about.
G3	**Engineer bottleneck.** Expert analysis does not scale linearly with fleet size [11].	S / A	Needs ranked, explained, abstaining triage, not more alerts.
G4	**Vendor and operator data silos.** Each OEM sees its own engines; operators see mixed fleets. Skywise is the main cross-fleet attempt [14].	S	Cold-start for new engine types (new models have few run-to-failure examples).
G5	**Snapshot legacy.** Standard EHM remains a handful of points per flight [5].	S	Early degradation signatures can hide between snapshots.
G6	**Censoring and removal bias.** Preventive removals mean *failures are rare labels*; naive supervised RUL is trained on a world that does not exist.	A	Hazard/survival formulation is the principled fix.
G7	**Regulatory readiness for learning-based components.**	S	Design must produce certification-style evidence, not just models.

	EASA’s AI guidance (Level 1 assistance, Level 2 human-AI teaming) requires learning assurance and explainability; the proposed Issue 03 was in public consultation until 12 August 2026 [17, 18, 19].		
G8	**Hard-limit interaction.** Life-limited parts are governed by approved cycle limits; ML prognostics may advise but must not silently override them.	A	The scheduler needs a hard-life guard.

3. Ten completely different solutions

Each solution is a different *paradigm*, not a different network. For each: the idea, how it would be built on C-MAPSS, its closest industry analogue, its fatal flaw, and a verdict on what to keep.

S1 · Deep sequence regressor (LSTM / GRU / TCN) — “the hackathon default”

Idea. Slide a 30–50-cycle window over the multivariate sensors; a recurrent or dilated-convolution network regresses a piecewise-linear capped RUL with MSE.

Industry analogue. None as a standalone product; neural models appear as *components* inside OEM monitoring (GE describes AI/ML engine models for 24×7 monitoring [4]).

Strengths. Quickest to build; strong on single-regime FD001/FD003; the hint in the brief.

Fatal flaw. Outputs one number with no error bar, assumes run-to-failure labels fleets do not have, trains on symmetric loss though cost is asymmetric, is opaque for certification, and loses several RMSE points on multi-regime data without per-regime normalisation (MEASURED with a gradient-boosting proxy: +2.3 on FD002, +3.2 on FD004; Section 7.1).

Verdict. Mandatory component; insufficient system. Roughly every team will submit this, so it is table stakes.

S2 · Health index + analog trajectory matching

Idea. Fuse sensors into a one-dimensional health index (PCA, regression or autoencoder); match the engine’s recent HI trajectory to a library of run-to-failure curves (DTW / k-NN); RUL is the distribution of the matches’ remaining lives.

Industry analogue. Closest to practice: baseline-and-delta trending toward a limit (EGT-margin style) and similarity-based fleet analytics [1].

Strengths. Interpretable; produces a natural spread of outcomes; data-light.

Fatal flaw. A 1-D index discards multi-fault structure; sensitive to operating regime; only 100–260 library curves per subset.

Verdict. Keep as a *diverse ensemble member* and as the engineer-facing “health curve”.

S3 · Physics gas-path model + particle / Kalman filter

Idea. Thermodynamic engine model with hidden health parameters (component efficiency, flow capacity) estimated by an EKF/UKF/particle filter; a degradation law propagates to a limit.

Industry analogue. Classical gas-path analysis, the long-standing core of OEM engine-health monitoring, and the “twin” in GE/Rolls-Royce language [1, 6].

Strengths. Physically meaningful, extrapolates, works with little data, easiest to certify.

Fatal flaw. Needs a validated engine model, which the C-MAPSS files do not ship; model mismatch and filter tuning dominate; cannot learn from the fleet.

Verdict. Infeasible in a hackathon as machinery. **Steal the idea**: healthy-baseline residuals and a monotone latent health state.

S4 · Self-supervised Transformer “foundation” model

Idea. Pre-train a patch-based Transformer by masked sensor reconstruction on all subsets and unlabeled windows; fine-tune a RUL head.

Industry analogue. OEM research programmes; not documented as production.

Strengths. Best published accuracy on multi-regime data (FD004 reported around 14–14.5 RMSE [24, 25, 26]); exploits unlabeled data.

Fatal flaw. With a few hundred engines the gap to a good CNN/GRU is inside bootstrap noise (Finding 2); no uncertainty, opaque, heavy at the edge.

Verdict. Include as one expert; do not bet the system on it.

S5 · Bayesian deep ensembles / deep Gaussian process

Idea. MC-dropout, deep ensembles or sparse GPs give a predictive distribution over RUL.

Industry analogue. CFM describes “probabilistic diagnostic and prognostic” ML for LEAP [12].

Strengths. Honest spread; epistemic signal for out-of-distribution detection.

Fatal flaw. Posterior or ensemble spread is *not* calibrated coverage; ensembles are typically over-confident under shift and cost 5–10× compute.

Verdict. Use ensembles for diversity; **wrap with conformal calibration** so coverage is a measured property.

S6 · Survival / hazard (time-to-event) model

Idea. Predict a discrete-time hazard h(k | x) with a censored likelihood, yielding a survival curve S(k) rather than a point.

Industry analogue. Fleet reliability engineering (Weibull-style); Airbus markets Reliability Services alongside predictive and health monitoring [13].

Strengths. Handles censoring natively; the hazard drops directly into an optimal-stopping maintenance rule (Section 6.4); honest about “not yet failed”.

Fatal flaw. Lower point accuracy than direct regression on C-MAPSS; needs enough events; discretisation choices matter.

Verdict. Essential head in the final system, the bridge from prognosis to decision.

S7 · Physics-informed hybrid digital twin (PINN / residual learning)

Idea. Neural network trained with physics-motivated losses (monotone degradation, conservation relations) or learning residuals on top of a physics model.

Industry analogue. GE and Rolls-Royce engine twins [1, 6].

Strengths. Plausibility and better extrapolation.

Fatal flaw. Needs trustworthy physics; many interacting loss weights; training instability; no engine model in the dataset.

Verdict. Adopt the *light* version: a monotone health bottleneck plus gas-path ratio features.

S8 · Unsupervised anomaly detection + change-point onset

Idea. Learn the healthy envelope (autoencoder, isolation forest, Mahalanobis); detect onset by CUSUM / Bayesian change-point; extrapolate the trend.

Industry analogue. The dominant production pattern: alerts from deviation against known operating signatures [12].

Strengths. Needs no failure labels, which matches real fleets (Gap G6); cheap; edge-friendly.

Fatal flaw. Detects but does not prognosticate; thresholds are arbitrary; RUL accuracy is poor.

Verdict. Use as *onset detector* and as the **out-of-distribution guard** in the trust envelope.

S9 · Federated / transfer fleet learning

Idea. Domain adaptation across regimes and fault modes, and federated averaging across operators, so no raw telemetry leaves an airline; few-shot adaptation per engine type.

Industry analogue. Skywise's cross-airline data platform [14]; OEMs centralising fleet data.

Strengths. Targets the real bottleneck: silos and cold start (G4).

Fatal flaw. Four simulated subsets cannot demonstrate it convincingly; privacy engineering is substantial; no decision layer.

Verdict. Roadmap item plus a cheap simulation (split engines into three "airlines", test cross-subset transfer).

S10 · Offline reinforcement learning / POMDP maintenance policy

Idea. Skip RUL; learn *continue / inspect / maintain* directly from a simulated environment with a cost reward.

Industry analogue. None in documented production; optimisation happens in contract and planning tools.

Strengths. Optimises the true objective; adapts to changing costs.

Fatal flaw. C-MAPSS contains no maintenance actions, so no counterfactuals: you must build a simulator from the same simulated data (circular); sample-hungry; uncertifiable.

Verdict. Replace with **optimal stopping on a calibrated distribution** (closed-form in Section 6.4) plus Monte-Carlo policy evaluation: most of the benefit, far less risk.

The cross-cutting lesson

Every solution optimises a single axis: S1 accuracy, S3 plausibility, S5 uncertainty, S6 censoring, S8 labels, S9 silos, S10 decisions. Real prognostics systems are *portfolios*. The final design composes the strengths and neutralises the flaws: S2/S3/S7-light for the health layer, S1/S4/S5 plus a gradient-boosting expert for the core, S6 as a head, S8 as the guard, S10-light for decisions, S9 as the learning loop.

4. Head-to-head comparison

4.1 Scored matrix

Scores (1 worst, 5 best) are **my judgement informed by the literature and by the measured results in Section 7; they are not experimental outputs.** Weights reflect a fleet-operator view: uncertainty and decision-readiness weigh as much as accuracy. The bracketed number is the weighted total out of 5. The final design scores highest *by construction* (it is the composite); its weak points are edge efficiency and 48-hour feasibility, which Section 9 addresses with tiers.

	Accuracy 15%	Uncertainty quality 15%	Censoring realism 10%	Regime/fault generalisation 10%	Explain-ability 12%	Decision readiness 15%	Robustness 8%	Edge efficiency 5%	48-h feasibility 10%
S1 Deep sequence regressor (LSTM/GRU/TCN) [2.28]	4	1	1	3	1	1	2	4	5
S2 Health-index + analog trajectory matching [3.17]	3	3	3	2	4	3	3	4	4
S3 Physics gas-path model + particle filter [3.39]	2	4	4	3	5	4	3	3	2
S4 Self-supervised Transformer foundation model [2.63]	5	1	3	4	2	1	3	2	3
S5 Bayesian deep ensembles / deep GP [3.03]	4	4	2	3	2	3	3	2	3
S6 Survival / hazard (time-to-event) model [3.70]	3	4	5	3	3	5	3	4	3
S7 Physics-informed hybrid digital twin [2.95]	4	2	2	4	4	2	4	3	2
S8 Unsupervised anomaly + change-point [2.88]	1	2	5	3	3	2	4	5	4
S9 Federated / transfer fleet learning [2.61]	3	2	3	5	2	1	4	3	2
S10 Offline RL / POMDP maintenance policy [2.28]	1	3	2	2	1	5	2	3	1
VISION-PHM (final, composite) [4.30]	**5**	**5**	**4**	**4**	**4**	**5**	**4**	**3**	**3**

Scoring of the ten candidate solutions and the composite final design

4.2 Capability matrix against industry

✔ = documented in public sources reviewed · ◐ = designed for the hackathon prototype (partial) · – = not found in the public sources reviewed (not a claim of absence).

Capability	GE	R-R	P&W	CFM / Safran	Airbus Skywise	VISION-PHM
Per-engine digital twin / replica	✔ [1]	✔ [6]	–	–	✔ (ops twin) [14]	◐ (health-index twin)
Continuous / full-flight data	✔ [1]	✔ [5]	✔ [10, 11]	multi-phase only [12]	✔ (ACARS-based) [13]	✔ (replay stream)
ML prognostics / early detection	✔ [4]	✔ [6]	✔ [11]	✔ [12]	✔ [13]	✔
Workscope / shop-visit forecasting	✔ [3]	✔ [6]	✔ [9]	–	–	◐ (decision layer)
Calibrated RUL distribution with coverage evidence	–	–	–	–	–	**✔ (measured, §7.2)**
Risk-controlled maintenance trigger	–	–	–	–	–	**✔ (measured, §7.4)**
Censoring-aware (hazard) training	–	–	–	–	–	✔ (design)
Abstain-and-escalate behaviour	–	–	–	–	–	✔ (design)
Certification-style evidence pack (EASA learning assurance)	–	–	–	–	–	◐ (templates)
Engineer-in-the-loop review	✔	✔ [5]	✔ [11]	✔	✔ [13]	✔

The bold rows, plus censoring-aware training and abstention, are where a prototype can credibly *exceed what the public record shows*. The caveat cuts both ways: incumbents may do these things internally. The claim is about the public record, and about what a hackathon team can demonstrate and evidence.

4.3 What each solution would have to beat

Solution	Closest incumbent practice	Where incumbent is stronger	Where the solution is stronger
S1	OEM ML monitoring	Real sensors, full-flight data, engineers	Nothing structural; marginal accuracy
S2	Baseline-delta trend to limit	Condition-corrected margins in operations	Distribution of analog outcomes
S3	Gas-path analysis / twins	Validated engine models	None without a model
S4	OEM research	Data scale	Multi-regime accuracy in benchmarks
S5	Probabilistic ML at CFM	Production hardening	Explicit predictive spread
S6	Fleet reliability services	Large failure histories	Per-engine, covariate-dependent hazard
S7	Engine twins	Proprietary physics	Flexibility if physics is imperfect
S8	Signature-based alerts	Curated signature libraries	Label-free onset on any sensor set
S9	Cross-airline platforms	Real data access	Privacy-preserving by design
S10	Contract-level optimisation	Real cost structures	End-to-end objective (in theory)

5. Flaw → improvement → innovation ledger

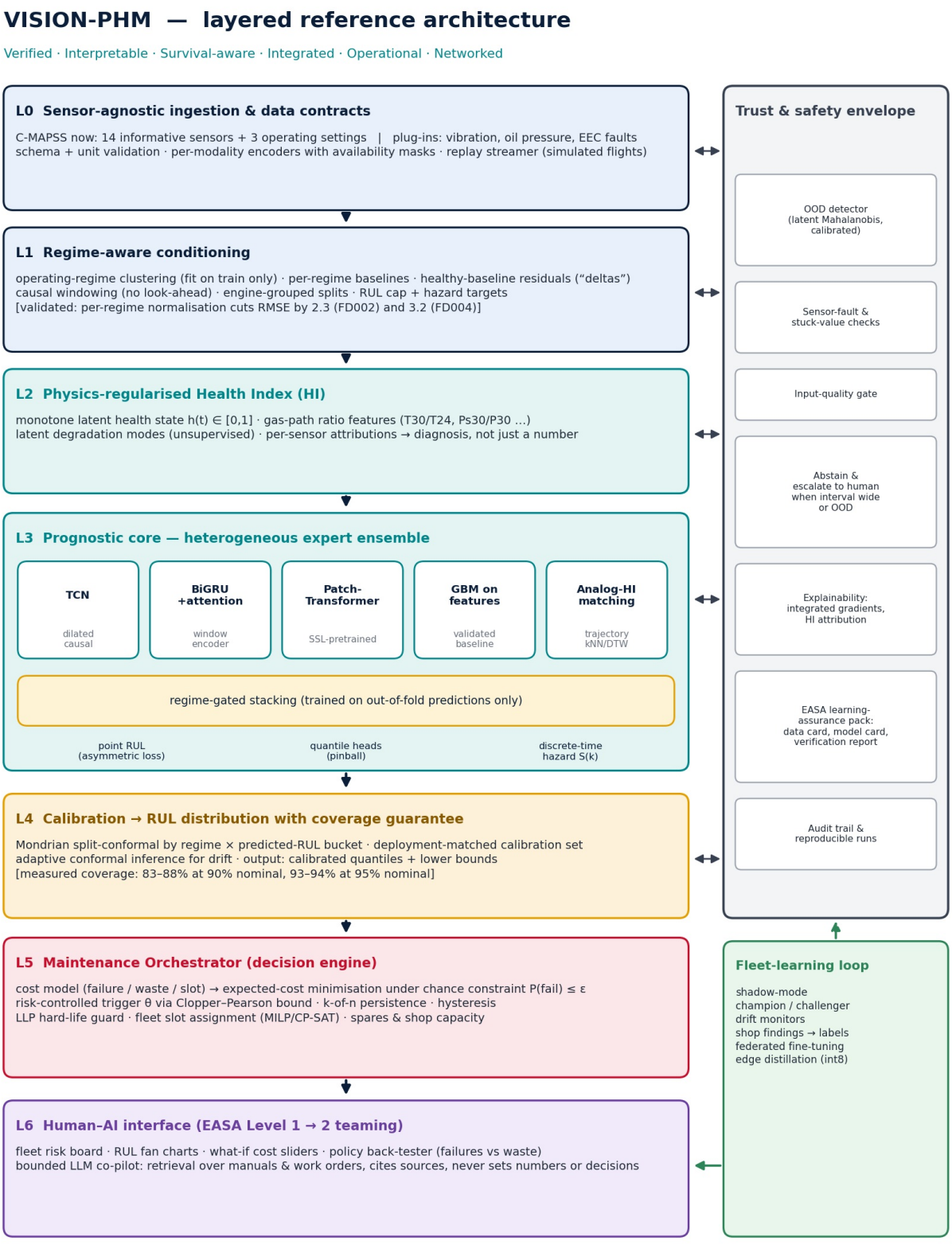
Every row of the final design traces to a flaw in one of the ten solutions or in current practice (G1–G8).

#	Flaw (source)	Improvement adopted	Innovation in VISION-PHM	Status
1	Point RUL cannot be scheduled (S1, G1)	Output a full RUL distribution	Quantile heads + conformal calibration (L4)	MEASURED
2	Ensemble spread ≠ coverage (S5, G2)	Distribution-free calibration	Mondrian (regime × predicted-RUL) split-conformal with *deployment-matched* calibration	MEASURED
3	Regime shifts hurt (S1, S4)	Per-regime normalisation, healthy-baseline residuals	Regime-gated stacking of experts	MEASURED / DESIGN
4	Labels are censored (G6, S1)	Time-to-event head	Hazard head feeds the cost rule directly	DESIGN
5	Thresholds tuned in-sample break risk targets (all point-RUL triggers)	Hold-out threshold selection	**Risk-controlled trigger** with Clopper–Pearson bound on failure rate	MEASURED
6	Alerts, not decisions (G1)	Cost-aware optimal stopping	Closed-form hazard-vs-cost rule; fleet slot assignment (MILP/CP-SAT)	DESIGN (rule) / MEASURED (policy sim)
7	Opaque models (S1, S4, G7)	Monotone health bottleneck + attributions	Per-sensor attribution on the health index; learning-assurance artefact pack	DESIGN
8	Engineer bottleneck (G3)	Ranked triage	Abstain-and-escalate when interval wide or OOD	DESIGN
9	Silos and cold start (G4, S9)	Transfer + federated adaptation	Three-“airline” federated simulation; cross-subset transfer test	DESIGN
10	No vibration / oil channels in C-MAPSS (trap 1)	Modality-agnostic encoders with masks	Channel-dropout robustness test; adapter spec for N-CMAPSS / bearing data	DESIGN
11	RMSE-only evaluation (trap 3, 4)	Bootstrap CIs, fixed protocol	**Decision-level back-testing** (failure rate vs unused life) as a first-class metric	MEASURED
12	ML must not override hard limits (G8)	Rule-based guard	Life-limit guard in the scheduler	DESIGN
13	Degradation depends on how the engine is flown [21]	Operating-history features (N-CMAPSS)	Mission-aware prognostics: advise derate / routing when risk budget is tight	DESIGN (stretch)
14	LLMs are fluent and unreliable	Constrain the role	Co-pilot retrieves manuals and drafts work orders; **never** sets numbers or decisions	DESIGN

6. The final solution: VISION-PHM

Verified · **I**nterpretable · **S**urvival-aware · **I**ntegrated · **O**perational · **N**etworked — *Prognostics and Health Management*

6.1 Reference architecture



VISION-PHM layered reference architecture

6.2 Design principles

1. **Decision-first.** Each layer exists to make the maintenance decision safer or cheaper; a metric that does not move a decision is secondary.
2. **Calibrated or silent.** No number leaves the system without a coverage statement; if coverage cannot be stated, the system abstains.
3. **Heterogeneous beats homogeneous.** Neural, tree, analog and hazard experts fail differently; the ensemble exploits that.

4. **Physics-light, not physics-free.** Monotone health, gas-path ratios and healthy-baseline residuals inject engineering knowledge without needing a validated engine model.
5. **Protocol before model.** Splits, normalisation scope and RUL cap are frozen on day one and tested for leakage.
6. **The human holds authority.** Default posture is EASA "Level 1" assistance; any automatic action requires the Level 2 evidence base [17, 18].
7. **Everything is back-testable.** A policy simulator replays held-out engines through the full decision chain.

6.3 Layer specification

L0 · Sensor-agnostic ingestion. Pydantic/JSON-schema data contracts; unit harmonisation; one encoder per modality (gas-path, speeds, vibration spectrum, oil system) with an *availability mask* so a missing channel degrades gracefully rather than crashing. For C-MAPSS: 14 informative sensors and 3 operating settings (Appendix B). A replay streamer emits one cycle per engine per tick to emulate a live feed. DESIGN

L1 · Regime-aware conditioning. Cluster the operating settings (k = 6 on FD002/FD004; fit on training data only); normalise each sensor *within* regime; optionally subtract a healthy-baseline regression (sensor as a function of operating condition, fitted on early-life cycles) to produce industry-style "delta" parameters. Causal windows only. RUL target capped at 125 (sweep 100–140); hazard targets built alongside. MEASURED (per-regime normalisation) / DESIGN (delta residuals; untested here)

L2 · Physics-regularised health index. A bottleneck network emits h(t) ∈ (0,1), trained with a monotonicity penalty and boundary conditions (Section 6.4b). Candidate gas-path ratio features (e.g. T30/T24, T50/T30; to be ablated) give the encoder thermodynamic hints. Unsupervised mixture heads capture latent degradation modes (FD003/FD004 contain two fault modes [20]). Integrated-gradient attributions over the encoder tell the engineer *which sensors drove the drop*. DESIGN

L3 · Prognostic core. Five heterogeneous experts, stacked per regime on **out-of-fold predictions only** (never on training-fit predictions):

Expert	Role	Starting configuration (to be tuned)
TCN	Fast causal feature extractor	4 residual blocks, 64 ch, kernel 3, dilations 1-2-4-8, dropout 0.2
BiGRU + attention	Window encoder (window is entirely past data, so bidirectionality is legitimate)	2 layers × 64, additive attention
Patch-Transformer	Cross-regime pattern capture	patch 5, d = 64, 3 layers, 4 heads; masked-patch pre-training (40% masking) on unlabeled windows
Gradient boosting	Diverse, cheap, strong tabular baseline	the configuration validated in Section 7
Analog-HI matcher	Trajectory-level evidence	k-NN/DTW on the L2 index against run-to-failure library

Three heads on the stacker: **point RUL** (Huber + asymmetric late-penalty), **quantile heads** (pinball loss at τ ∈ {0.05, …, 0.95}), **discrete-time hazard** (censored likelihood). Optimiser: AdamW, cosine schedule, early stopping on engine-grouped validation, three seeds. DESIGN (neural experts) / MEASURED (gradient-boosting expert)

L4 · Calibration. Mondrian split-conformal intervals, bucketed by regime × predicted-RUL, on an engine-grouped calibration set **sampled the way the model is used** (the hazard of mismatch is measured in Section 7.2). Adaptive conformal inference tracks drift online by nudging the miscoverage level α after each outcome. Output: calibrated quantiles and one-sided lower bounds. MEASURED (static); DESIGN (adaptive)

L5 · Maintenance Orchestrator. (i) *Cost model*: in-service failure C_f, unused-life cost c_w per cycle, slot and logistics costs. (ii) *Decision rule*: expected-cost minimisation with a chance constraint (Section 6.4e). (iii) *Trigger*: lower-bound threshold selected by the risk-controlled procedure (6.4f) with k-of-n persistence to suppress flapping. (iv) *Hard-life guard*: approved life limits always take precedence. (v) *Fleet layer*: assign engines to hangar slots with a MILP/CP-SAT model under capacity, spares and lead-time constraints. MEASURED (trigger and policy simulation) / DESIGN (fleet layer)

L6 · Human–AI interface. Fleet risk board (ranked by P(fail within lead time)); engine view with health curve, RUL fan chart and sensor attribution; what-if cost sliders; policy back-tester; a bounded **LLM co-pilot** that retrieves manual passages and past work orders and drafts a work-order text with citations, but never produces numbers or schedules. DESIGN

Trust and safety envelope (cross-cutting). Out-of-distribution score (Mahalanobis distance in the L2 latent space, threshold calibrated on validation engines); sensor-fault detectors (stuck, out-of-range, spike); input-quality gate; **abstain-and-escalate** when the interval is wider than a configured maximum or an OOD flag is raised; an EASA-style learning-assurance pack (data card documenting completeness and representativeness, model card, verification report) [17, 18, 19]; reproducible, hashed runs. DESIGN

Fleet-learning loop. Shadow-mode deployment, champion/challenger comparison, drift monitors (population-stability index on regime-conditioned residuals), shop findings converted into labels, federated fine-tuning across operators, distillation of a small int8 TCN for ground-station triage. "Edge" here means an airline ground station or gateway, **not** certified on-engine FADEC software. DESIGN

6.4 The mathematical core

(a) Asymmetric training loss. With capped target y = min(RUL, R_{max}) and error d = ŷ − y (positive = late warning):

```
L_reg = Huber_δ(d) + λ · max(d, 0)²          # over-estimates (late) cost more
NASA score  s = Σ_i  exp(−d_i/13) − 1  if d_i < 0   else   exp(d_i/10) − 1
```

(b) Monotone health index. With $\bar{h}$ the exponentially smoothed index:

```
L_HI = (1/T) Σ_t ReLU( h̄_{t+1} − h̄_t )  +  β·[ (h_early − 1)² + (h_end − 0)² ]
```

(c) Hazard head with censoring. $h_k(x) = \sigma(f_\theta(x, k))$; $S(k) = \prod_{j \le k}(1 - h_j)$.

```
failed at k_e :  log h_{k_e} + Σ_{j<k_e} log(1 − h_j)
censored at k_c:               Σ_{j≤k_c} log(1 − h_j)
```

(d) Mondrian conformal lower bound. For bucket b with n_b calibration residuals $r_i = \hat{y}_i - y_i$: q_b is the $\lceil (n_b+1)(1-\alpha) \rceil$-th smallest residual, and LB = ŷ − q_b, giving P(y ≥ LB) ≥ 1 − α when calibration and test engines are exchangeable within the bucket. Samples within one engine are

dependent, so calibration must be engine-grouped and the guarantee is *marginal over engines*, not per time step.

(e) Optimal stopping on the predicted distribution. Let F be the predicted RUL CDF, C_f the failure cost, c_w the cost of one wasted cycle:

```
J(s) = C_f · F(s)  +  c_w · ∫_s^∞ (1 − F(u)) du        (+ slot costs)
dJ/ds = (1 − F(s)) · [ C_f · hazard(s) − c_w ]
⇒  remove at the first s where   hazard(s) = f(s)/(1 − F(s))  ≥  c_w / C_f
```

If a failure is judged 1,000× as costly as a wasted cycle, act when the forecast per-cycle failure hazard reaches 0.1%. This is a newsvendor/age-replacement result and is exact when the predictive hazard is increasing.

(f) Risk-controlled trigger. For candidate threshold θ, count k in-service failures among n held-out calibration engines; compute the one-sided Clopper–Pearson bound $UB(\theta) = \text{Beta}^{-1}(\text{conf}; k+1, n-k)$; choose the θ with least unused life among those with $UB(\theta) \le \varepsilon$. Tuning θ on the same data without the bound is shown in Section 7.4 to break the target frequently.

6.5 Leakage-safe data and training protocol

1. Split **by engine**; never by row. Fit normalisers, regime clusters and baselines on training engines only.
2. Stack and calibrate on out-of-fold predictions; never refit the final model on the calibration engines without re-deriving calibration [23].
3. Evaluate on the official last window per test engine against the official RUL file; also report the full-trajectory view.
4. Report capped-target results ($R_{max} = 125$) *and* sensitivity to the cap.
5. Unit-test causality: perturbing future cycles must leave a prediction unchanged.
6. Report bootstrap confidence intervals over test engines and three training seeds.

6.6 Deployment topology and scale

Tier	Contents	Notes
Stream	Kafka/Redpanda topic per fleet; replay producer in the demo	Ordering per engine key
Feature service	Causal windowing, regime assignment	O(window) state per engine
Model service	ONNX Runtime/Triton; full ensemble; small distilled TCN for triage	CPU is enough for the demo
Calibration & decision	Conformal tables, risk-controlled thresholds, scheduler	Stateless, versioned
Storage	Time-series store + model/calibration registry	Hashed artefacts for audit

Back-of-envelope: a 10,000-engine fleet at two flights per day is about 20,000 window inferences per day, trivial for CPU inference; data engineering and governance, not model compute, dominate. DESIGN (not benchmarked here)

7. Evidence from real C-MAPSS data

7.0 What was run, and what was not

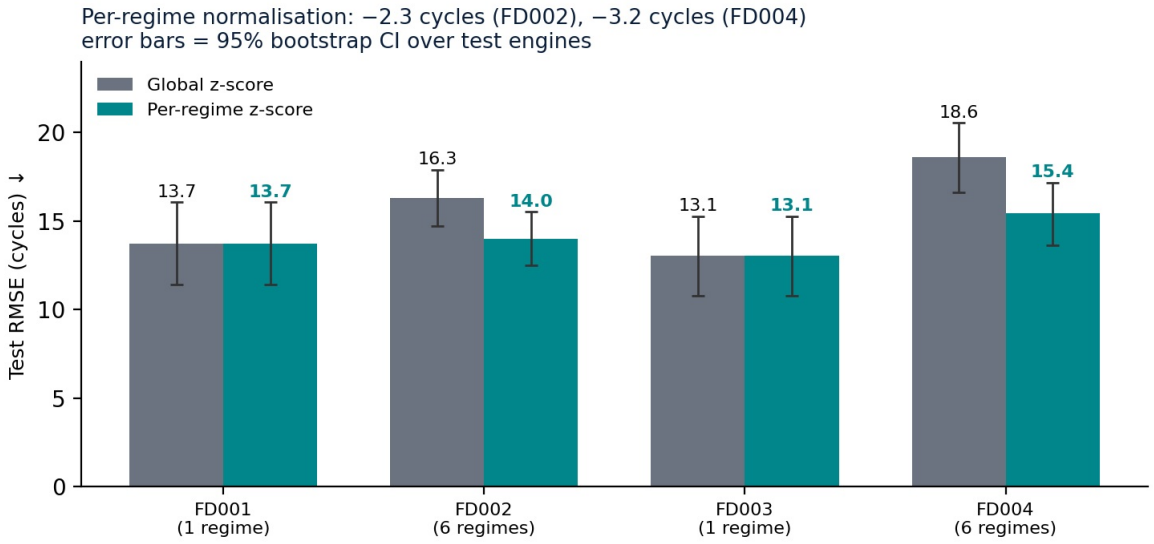
I downloaded the four official C-MAPSS subsets and ran everything in the supplied script `cmapss_validation.py`. The sandbox had no deep-learning framework, so **no neural network was trained**. The prognostic model used throughout is a *gradient-boosting regressor* on 57 causal window features (for each of 14 sensors: raw value, 5-cycle mean, 5-vs-30-cycle delta, 10-cycle standard deviation; plus the cycle count), trained on capped RUL (125). Its purpose is to test **the protocol, calibration and decision claims**, which are model-agnostic. It is *not* offered as a state-of-the-art RUL model, and the deep experts' accuracy in Section 6 is a target, not a result.

For decision experiments I used FD001 (easiest, 100 engines) and FD004 (hardest, 249 engines, 6 regimes, 2 fault modes). Predictions for every cycle of every training engine come from **5-fold engine-grouped out-of-fold** models. A trigger fires when its statistic crosses a threshold; the engine is then removed after a 15-cycle logistics lead time; if that removal is not before the engine's failure cycle, it counts as an in-service failure.

7.1 Protocol effect and statistical noise

Subset	Normalisation	Test RMSE [95% CI]	NASA score	Paired ΔRMSE vs global [95% CI]
FD001	single regime	13.72 [11.4, 16.1]	298	n/a
FD002	global z-score	16.31 [14.7, 17.9]	1310	
FD002	**per-regime z-score**	14.01 [12.5, 15.5]	896	**−2.30** [−3.68, −0.94]
FD003	single regime	13.05 [10.8, 15.3]	313	n/a
FD004	global z-score	18.59 [16.6, 20.5]	2168	
FD004	**per-regime z-score**	15.42 [13.6, 17.2]	1282	**−3.18** [−4.62, −1.79]

Gradient-boosting proxy, 14 sensors, capped RUL 125, last window of each official test engine; bootstrap over test engines (2,000 resamples; paired 4,000).



Effect of per-regime normalisation, with 95% bootstrap CIs

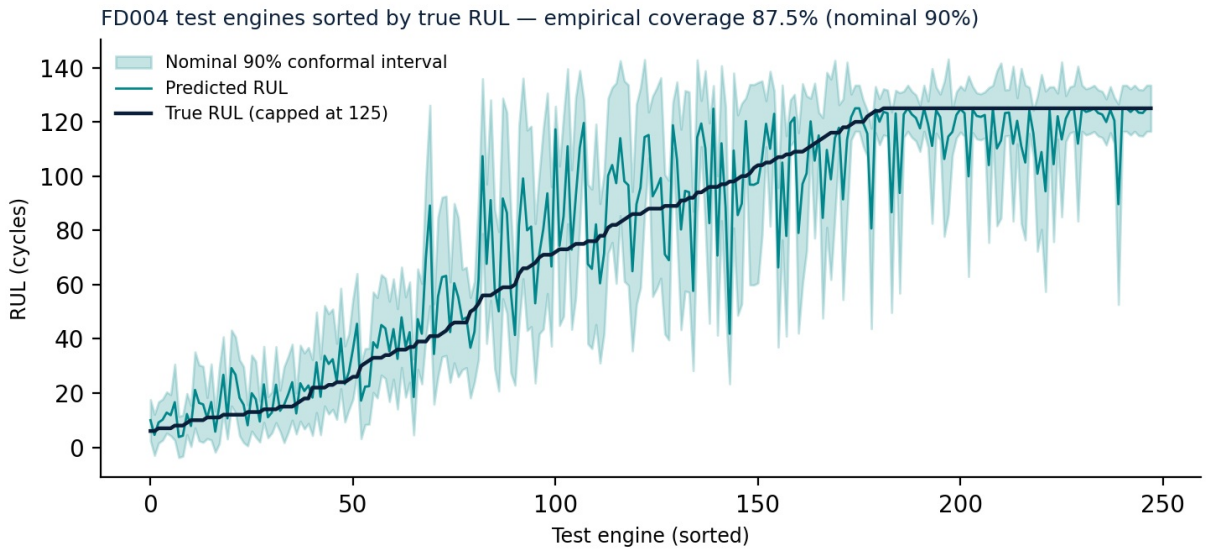
MEASURED Per-regime normalisation lowers RMSE by 2.3 cycles on FD002 and 3.2 on FD004; the paired bootstrap intervals exclude zero, so the effect is real, in the same direction as the protocol study in [23]. Separately, single-model RMSE intervals are about ±2.3 cycles wide on the 100-engine subsets: **published gaps of roughly one cycle on FD001/FD003 cannot be distinguished from sampling noise.** *Literature, not re-run and not protocol-matched* LITERATURE: recent papers report about 11–12.6 (FD001), 12–13 (FD002), 11–12.7 (FD003), 14–14.5 (FD004) [24, 25, 26, 27]; an early LSTM reports 16.1 / 24.5 / 16.2 / 28.2 [24]. The proxy model sits within one to three RMSE of the best reported values.

7.2 Calibration: helpful, but not automatic

Subset (test engines)	Nominal	Empirical coverage	Mean width (cycles)	Naive global interval at 90%
FD001 (100 engines)	80%	75.0%	30.0	
	90%	83.0%	37.6	coverage 92%, width 46.1
	95%	93.0%	48.0	
FD004 (248 engines)	80%	81.5%	30.6	
	90%	87.5%	40.1	coverage 87%, width 48.3
	95%	94.4%	53.6	

Mondrian split-conformal on out-of-fold residuals (buckets by predicted RUL), evaluated on the last window of each official test engine against

RUL capped at 125. One-sided 90% lower bound held for 86% (FD001) and 87% (FD004) of engines; 95% bound for 96% and 94%.



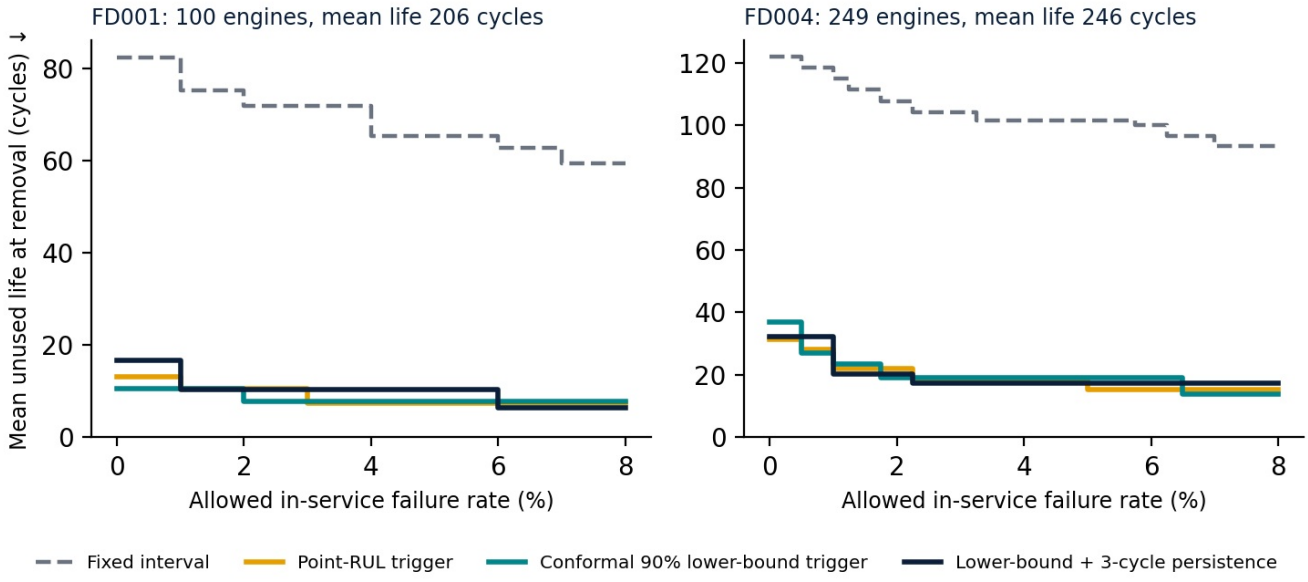
FD004 test engines sorted by true RUL, with nominal 90% Mondrian conformal band

MEASURED Coverage ranged from 7 points below nominal (FD001 at 90%: 83%) to 1.5 points above (FD004 at 80%); at 90% nominal it was 83% on FD001 and 87.5% on FD004, and at 95% it was 93% and 94%. Calibration was done on out-of-fold residuals pooled over all cycles, but evaluation is on one last window per engine, a different sampling distribution, so exchangeability is only approximate. With only 100 test engines on FD001, a coverage estimate also carries about ±4 points of sampling noise. On FD004 the bucketed intervals are about 17% narrower than a single global interval at essentially the same coverage (40.1 vs 48.3 cycles; 87.5% vs 87.1%); on FD001 the comparison is confounded by different coverage. **Design consequence:** calibration data must mimic deployment sampling, and online adaptive conformal inference is required, not optional. The interval width itself (about 40 cycles at 90%) is the honest statement of how well a 125-cycle horizon can be predicted.

7.3 Decision value: prognosis versus calendar

Dataset	Policy	≤0% failures	≤1%	≤2%	≤5%
FD001 (mean life 206)	Fixed interval	82.3	75.1	71.8	65.3
	Point-RUL trigger	13.2	10.4	10.4	7.3
	Conformal 90% lower bound	10.5	10.5	7.8	7.8
	Lower bound + 3-cycle persistence	16.7	10.3	10.3	10.3
FD004 (mean life 246)	Fixed interval	122.0	114.9	107.8	101.5
	Point-RUL trigger	31.5	22.0	22.0	15.3
	Conformal 90% lower bound	36.9	23.5	19.1	19.1
	Lower bound + 3-cycle persistence	32.2	20.2	20.2	17.2

Cell = mean unused life at removal (cycles) of the best threshold whose failure rate is within the budget; thresholds chosen on the same engines, so values are optimistic for every policy alike. Lead time 15 cycles; out-of-fold predictions.



Mean unused life at removal vs allowed in-service failure rate

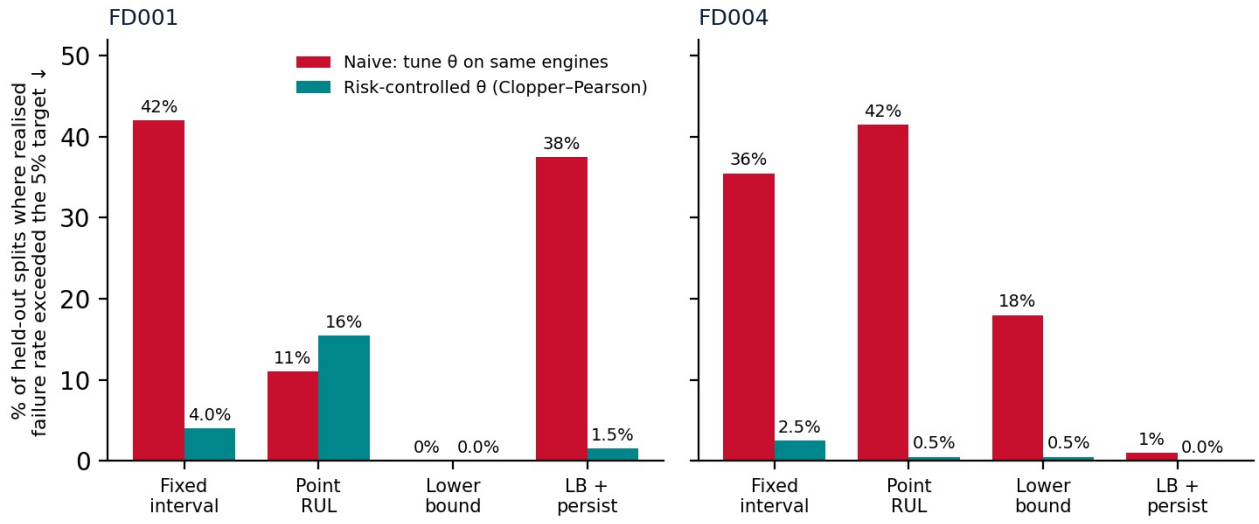
MEASURED At every failure budget tested, any prognostic trigger leaves far less unused life than the best fixed removal age: at a 1% budget, FD001 drops from about 75 cycles to about 10 cycles and FD004 from about 115 cycles to about 20–23 cycles. **The conformal lower-bound trigger is not a uniform winner on unused life**: it is better at some budgets (FD001 at 0% and 2%; FD004 at 2%) and worse at others (FD004 at 0% and 5%). The advantage over fixed intervals is smallest at a 0% budget on FD004 (about 3×) and largest at moderate budgets (up to about 9×). What the lower bound buys, as the next experiment shows, is predictability.

7.4 Held-out threshold selection: the finding that changed the design

Thresholds were selected on a random half of the engines to meet a **5% failure target**, then applied to the other half, repeated 200 times. "Naive" picks the cheapest threshold whose *observed* failure rate on the selection half is ≤ 5%. "Risk-controlled" requires the one-sided Clopper–Pearson 90%-confidence upper bound to be ≤ 5% (Section 6.4f).

Dataset	Policy	Selection	Realised failure rate	Splits breaking 5% target	Unused life (cycles)
FD001	Fixed interval	naive	5.2%	42.0%	66.8
		controlled	1.6%	**4.0%**	78.3
	Point-RUL trigger	naive	3.0%	11.0%	7.7
		controlled	1.7%	**15.5%**	11.3
	Conformal 90% lower bound	naive	1.9%	0.0%	7.8
		controlled	1.1%	**0.0%**	9.8
	Lower bound + 3-cycle persistence	naive	3.9%	37.5%	8.9
		controlled	1.0%	**1.5%**	13.9
FD004	Fixed interval	naive	4.5%	35.5%	100.5
		controlled	2.0%	**2.5%**	107.4
	Point-RUL trigger	naive	4.3%	41.5%	16.5
		controlled	1.7%	**0.5%**	20.4
	Conformal 90% lower bound	naive	2.7%	18.0%	18.1
		controlled	1.5%	**0.5%**	20.5
	Lower bound + 3-cycle persistence	naive	2.1%	1.0%	17.1
		controlled	1.6%	**0.0%**	18.7

200 random 50/50 engine splits per cell; failure target ε = 5%; one-sided Clopper–Pearson confidence 90%.



Share of held-out splits in which the realised failure rate exceeded the 5% target

MEASURED Naive tuning broke the 5% target in **up to 42%** of held-out splits (fixed interval and point-RUL triggers). Risk-controlled selection reduced violations to at most 4% in 7 of 8 policy/dataset cases, at a cost of roughly 2–5 extra unused cycles for the prognostic triggers (about 1–2.5% of an engine's life). Two honest caveats: (i) **the exception**: the FD001 point-RUL trigger got *worse* (11% → 15.5%), because with only 50 engines per half a single missed failure flips the outcome; (ii) the conformal lower bound alone, even naively tuned, was the most reliable policy on FD001 (0% violations), and lower-bound-plus-persistence was the best controlled policy on FD004 (0% violations, 18.7 unused cycles vs 107 for the fixed interval). **Uncertainty did not make the triggers cheaper; it made their risk predictable**, which is what a safety case needs.

7.5 What these results do and do not establish

- **Established (on simulated C-MAPSS):** per-regime normalisation matters; RMSE differences below about 2 cycles are noise on 100-engine sets; prognostic triggers beat fixed intervals by about 5–8× in unused life on held-out engines; naive in-sample threshold tuning is unsafe; risk-controlled thresholds largely repair it.
- **Not established:** that a deep ensemble reaches state-of-the-art accuracy (not trained here); that the hazard head, health-index bottleneck, adaptive conformal inference, abstention logic or fleet scheduler work (designed, untested); anything about vibration or oil-pressure channels; real-fleet behaviour (C-MAPSS is simulated, run-to-failure and has no maintenance actions).
- **Statistical limits:** one model family and seed for the proxy; resampled hold-out splits reuse the same engines, so the violation percentages describe resampling variability rather than independent trials; the 15-cycle lead time and the threshold grids are assumptions; failure means reaching the simulated end-of-life cycle with no safety margin.

8. Evaluation scorecard

Tier	Metric	Prototype target	Baseline from this work
A	Test RMSE per subset (+95% bootstrap CI, 3 seeds)	At or below the proxy; stretch toward published values	13.7 / 14.0 / 13.1 / 15.4 (FD001–004)
A	NASA score per subset	Report alongside RMSE; lower is better	298 / 896 / 313 / 1282
B	Interval coverage at 80 / 90 / 95% nominal	Within ±3 points after deployment-matched calibration	75–82 / 83–88 / 93–94% (static, pooled calibration)
B	Mean interval width	Report; narrower at equal coverage is better	≈ 40 cycles at 90%
C	Unused life at removal vs in-service failure rate	≥ 5× better than fixed interval; **hold-out validated**	5–8× (Section 7.3–7.4)
C	Violation rate of risk target on held-out splits	≤ 5%	0–4% (7 of 8 cases)
C	False-alarm rate and warning lead time	Report per 1,000 flight cycles	Not measured here
D	RMSE under 20% random channel dropout	Graceful degradation curve	Not measured here
D	p95 inference latency per engine on CPU	< 100 ms for the full ensemble (design target)	Not measured here
D	Abstention rate and accuracy-on-retained	Report the trade-off curve	Not measured here

Ablation matrix to publish (remove one element at a time): per-regime normalisation → ensemble vs single expert → quantile heads → conformal → persistence → risk-controlled threshold → health-index bottleneck → hazard head.

9. Hackathon execution plan

9.1 Roles and timeline (48 hours, four people)

Roles: **A** data and protocol · **B** neural experts and ensemble · **C** uncertainty and decision engine · **D** dashboard, streaming and demo.

Hours	Milestone	Owner	Exit criterion
0–4	Data loaders, leakage-safe splits, causality unit test, regime clustering	A	Reproduces the proxy numbers in Section 7.1
4–14	GBM + TCN + BiGRU experts; asymmetric loss; out-of-fold pipeline	B (A assists)	Out-of-fold predictions for all four subsets
8–20	Mondrian conformal; deployment-matched calibration; coverage report	C	Coverage table with CIs
14–26	Stacking; health-index bottleneck; hazard head	B	Ablation table started
20–32	Policy simulator; risk-controlled thresholds; cost sliders; fleet scheduler (greedy first)	C	Pareto + held-out violation figure
12–36	Streaming replay, model service, dashboard (risk board, fan chart, attribution, what-if)	D	Live replay of 20 engines
32–42	Robustness: channel dropout, noise, cross-subset transfer, three-“airline” federated simulation	A/B	Robustness curves
42–48	Freeze, rehearse, record backup demo, write the evidence README	All	Demo runs offline

9.2 Scope tiers

- **Tier 1 (must, by hour 30):** protocol, per-regime normalisation, GBM + one neural expert, conformal intervals, policy simulator, dashboard with fan chart.
- **Tier 2 (should, by hour 40):** health index, hazard head, risk-controlled thresholds, attribution, abstention, greedy fleet scheduler.
- **Tier 3 (could):** Transformer pre-training, analog-HI expert, federated simulation, LLM co-pilot, mission-aware features from N-CMAPSS [21, 22].

9.3 Five-minute demo script

1. **Hook (30 s):** “Every team will show you an RMSE. We show you the maintenance decision and the evidence that it is safe.”
2. **Stream (60 s):** replay 20 engines; the risk board re-ranks live; open one engine: health curve, RUL fan chart, sensor attribution.
3. **Decision (90 s):** move the cost slider; the schedule moves; show the back-tested Pareto against fixed-interval maintenance (5–8× less unused life at equal risk).
4. **Honesty (60 s):** show the naive-vs-risk-controlled violation chart and the coverage table, “including where we under-cover”.
5. **Trust (60 s):** inject a stuck sensor; the system flags it and abstains; show the learning-assurance pack outline.
6. **Close (30 s):** the roadmap: vibration/oil adapters, N-CMAPSS, federated fleets.

9.4 Questions to be ready for

Question	Short answer
“Why not just a Transformer?”	On 100-engine test sets the accuracy gap is inside bootstrap noise (Section 7.1); the value is in calibration and decisions.
“The brief mentions vibration and oil pressure.”	C-MAPSS has neither; the architecture takes them through masked modality adapters; we demonstrate robustness to missing channels and name N-CMAPSS and bearing datasets for that branch.
“Is simulated data credible?”	It is a benchmark, not proof. We state the limits (Section 7.5) and test protocol-level claims that transfer.
“Can this be certified?”	We target EASA Level 1 assistance and produce learning-assurance artefacts; Level 2 needs the fuller evidence base [17, 18].
“What if the model is wrong?”	Calibrated intervals, abstention, hard-life guard, human authority, shadow-mode rollout.

9.5 Repository layout

```
vision-phm/
  data/           loaders, splits, regime clustering, causality tests
  models/         gbm.py  tcn.py  gru_attn.py  patch_tf.py  analog_hi.py  stack.py
  uncertainty/    mondrian_conformal.py  adaptive_conformal.py  ood.py
  decision/       cost_model.py  trigger.py  scheduler.py  simulator.py
  serving/        stream_replay.py  model_service.py  api.py
  ui/             dashboard (risk board, fan chart, attributions, what-if)
  evidence/       cmapss_validation.py  results.json  figures/  model_card.md
```

9.6 Risk register

Risk	Likelihood	Mitigation
Leakage inflates results	Medium	Engine-grouped splits, causality unit test, protocol frozen at hour 4
Deep experts underperform the GBM	Medium	The stacker keeps the GBM; report honestly
Calibration under-covers	High (measured)	Deployment-matched calibration, adaptive conformal, report the gap
Scope creep	High	Tier gates at hours 30 and 40
Demo fails live	Medium	Offline recorded backup; replay data bundled

10. Limitations and the path to a real fleet

1. **C-MAPSS is simulated, run-to-failure and action-free.** Real data are censored by maintenance, contaminated by washes, repairs and shop visits that *reset* health, and affected by sensor calibration drift. A production system needs explicit maintenance-event handling that restores health and reopens the baseline.
2. **Vibration and oil-system channels are untested here.** Their usefulness for specific failure modes must be shown on data that contains them.
3. **Calibration is only as good as exchangeability.** Fleet drift, new engine variants and operating changes break it; adaptive conformal inference mitigates but does not remove the risk.
4. **Regulatory path.** Ground-based decision support corresponds to EASA's Level 1 guidance; moving to Level 2 or beyond requires the full learning-assurance and explainability evidence, and the guidance is still evolving (Issue 03 consultation closed on 12 August 2026) [17, 18, 19].
5. **No on-engine claim.** Nothing here is intended for certified FADEC software.
6. **Costs are parameters, not facts.** The cost ratio drives the optimal trigger; real contracts (flat-rate, power-by-the-hour) would define it.

Moonshot horizon *(speculative, not part of the prototype):* mission-aware prognostics that recommend derate or routing choices when the risk budget is tight, building on the operating-history dependence of N-CMAPSS degradation [21]; formal robustness certificates for time-series models; federated conformal calibration across operators; and a whole-MRO-network counterfactual simulator extending the workscope forecasting that OEMs already do [3, 4].

Appendix A. Reference code

The snippets below are the tested core of Sections 6.4d–f (they reproduce the logic in `cmapss_validation.py`).

```
import numpy as np
from scipy.stats import beta, norm

EDGES = [15, 30, 50, 75, 100, 115]              # predicted-RUL bucket edges (cycles)
bucket = lambda p: np.digitize(p, EDGES)

def conformal_lower_bounds(pred_cal, y_cal, pred_new, alpha=0.10, min_n=30):
    """One-sided Mondrian split-conformal lower bound on RUL."""
    r_all = pred_cal - y_cal
    def q(r):
        n = len(r); k = min(n, int(np.ceil((n + 1) * (1 - alpha))))
        return np.sort(r)[k - 1]
    qg, b_cal = q(r_all), bucket(pred_cal)
    qb = {b: (q(r_all[b_cal == b]) if (b_cal == b).sum() >= min_n else qg)
          for b in range(len(EDGES) + 1)}
    return pred_new - np.array([qb[b] for b in bucket(pred_new)])

def cp_upper(k, n, conf=0.90):
    """One-sided Clopper-Pearson upper bound on a failure probability."""
    return 1.0 if k >= n else float(beta.ppf(conf, k + 1, n - k))

def choose_threshold(fail, waste, grid, eps=0.05, conf=0.90):
    """fail[g, e] in {0,1}; waste[g, e] = unused cycles (nan where failed)."""
    best, best_w = None, np.inf
    for g, th in enumerate(grid):
        k, n = int(fail[g].sum()), fail.shape[1]
        if cp_upper(k, n, conf) <= eps and (fail[g] == 0).any():
            w = np.nanmean(waste[g])
            if w < best_w: best, best_w = th, w
    return best

def optimal_removal(levels, values, cost_ratio=1e-3, horizon=125):
    """Remove at the first cycle s where the predictive hazard f(s)/(1-F(s)) >= c_w/C_f.
    A Gaussian is fitted to the predicted quantiles so the lower tail (where the
    decision lives) is smooth; swap in a skewed family if residuals warrant."""
    z = norm.ppf(levels)
    sigma, mu = np.polyfit(z, values, 1)                # values ~ mu + sigma * z
    s = np.arange(0, horizon + 1)
    zs = (s - mu) / sigma
    h = norm.pdf(zs) / (sigma * np.clip(norm.sf(zs), 1e-12, None))
    idx = np.nonzero(h >= cost_ratio)[0]
    return int(s[idx[0]]) if len(idx) else None
```

Appendix B. C-MAPSS cheat-sheet

Subset	Train engines	Test engines	Operating conditions	Fault modes
FD001	100	100	1	HPC degradation
FD002	260	259	6	HPC degradation
FD003	100	100	1	HPC + fan degradation
FD004	249	248	6	HPC + fan degradation

Sources: engine counts and conditions as tabulated in [26]; fault-mode descriptions per [20].

Sensor	Quantity	Used	Note
s2, s3, s4	T24, T30, T50: total temperature at LPC outlet, HPC outlet, LPT outlet	✔	T50 is the closest analogue to exhaust-gas temperature
s7, s11, s12	P30 (HPC outlet pressure), Ps30 (static), phi (fuel flow / Ps30)	✔	
s8, s9, s13, s14	Nf, Nc (physical fan/core speed), NRf, NRc (corrected)	✔	Spool speeds
s15, s17, s20, s21	Bypass ratio, bleed enthalpy, HPT/LPT coolant bleed	✔	
s1, s5, s6, s10, s16, s18, s19	T2, P2, P15, epr, farB, Nf_dmd, PCNfR_dmd	✘	Constant in FD001; dropped in all subsets (standard practice)
none	**Vibration, oil pressure**	—	**Not present in C-MAPSS**

Appendix C. Reproducing Section 7

1. Download the four C-MAPSS subsets (`train_FD00x.txt`, `test_FD00x.txt`, `RUL_FD00x.txt`) from the NASA Prognostics Data Repository, “Turbofan Engine Degradation Simulation Data Set”, and place them in one folder.
2. `pip install numpy pandas scikit-learn scipy`
3. `CMAPSS_DIR=./cmapss RESULTS_JSON=results.json python cmapss_validation.py` (about 5 minutes on a single CPU core).
4. The script is deterministic for a fixed library version; bootstrap and resampling seeds are set inside.

References

Public sources reviewed for this document (accessed 30 October 2026).

1. Inside Indiana Business, “Invisible digital twins taking flight at GE Aviation”: https://www.insideindianabusiness.com/articles/invisible-digital-twins-taking-flight-at-ge-aviation
2. PHM Society 2024, product showcase (GE Aerospace Fleet Monitor X): https://phm2024.phmsociety.org/?p=1605
3. GE Aerospace, Annual Report (AI material assistant, LEAP workscope nine months ahead): https://geaerospace.com/investor-relations/annual-report
4. GE Aerospace, AI fact sheet (2024): https://geaerospace.com/sites/default/files/ai-fact-sheet.pdf
5. Rolls-Royce, Engine Health Monitoring: https://www.rolls-royce.com/products-and-services/civil-aerospace/services/engine-health-monitoring.aspx
6. Rolls-Royce, Digital platforms (IntelligentEngine): https://www.rolls-royce.com/innovation/digital/digital-platforms.aspx
7. IFS blog, Rolls-Royce IntelligentEngine / Blue Data Thread: https://blog.ifs.com/?p=25191
8. Plant Services, Rolls-Royce IntelligentEngine data pipeline: https://www.plantservices.com/technology/artificial-intelligence/article/33015653/atownshend@endeavorb2b.com
9. RTX / Pratt & Whitney, EngineWise Intelligence: https://www.rtx.com/prattwhitney/services/enginewise/intelligence
10. Teledyne Controls, partnership with Pratt & Whitney (2021): https://www.teledynecontrols.com/news-events/news-detail?ID=39
11. Asian Aviation, “Pratt & Whitney has launched EngineWise”: https://asianaviation.com/pratt-whitney-has-launched-enginewise/
12. Safran, “CFM advances LEAP engine health monitoring” (10 April 2024): https://www.safran-group.com/pressroom/cfm-advances-leap-engine-health-monitoring-2024-04-10
13. Aviation Maintenance (AVM), Airbus Skywise Health Monitoring with Allegiant Air: https://www.avm-mag.com/?p=28558
14. Bernard & Hoffmann, “Skywise: big data platform as a foundation of airlines predictive and health monitoring”, PHM Asia-Pacific 2023: https://papers.phmsociety.org/index.php/phmap/article/download/3722/2187
15. AVM, digital twins in aviation (Lufthansa Technik AVIATAR): https://www.avm-mag.com/?p=32109
16. AVM, “Predictive engine maintenance: fix it before it’s broke” (MTU, P&W): https://avm-mag.com/predictive-engine-maintenance-fix-it-before-its-broke
17. EASA, AI Concept Paper proposed Issue 02, news item: https://www.easa.europa.eu/en/newsroom-and-events/news/easa-artificial-intelligence-concept-paper-proposed-issue-2-open
18. EASA, AI Concept Paper proposed Issue 03 (consultation to 12 Aug 2026): https://www.easa.europa.eu/newsroom-and-events/news/easa-releases-latest-issue-its-concept-paper-artificial-intelligence
19. Aviation Week, “Certifying artificial intelligence” (learning-assurance W-shaped process): https://aviationweek.com/forum/aerospace-defense/certifying-artificial-intelligence-key-automating-air-mobility
20. Saxena, Goebel, Simon, Eklund (2008), “Damage propagation modeling for aircraft engine run-to-failure simulation” (C-MAPSS); cited in PHM Society papers, e.g. https://papers.phmsociety.org/index.php/phmap/article/view/3690
21. Arias Chao, Kulkarni, Goebel, Fink (2021), “Aircraft engine run-to-failure dataset under real flight conditions”, *Data* 6(1):5 (N-CMAPSS): https://mdpi.com/2306-5729/6/1/5
22. PHM Society 2021 Data Challenge (N-CMAPSS): https://data.phmsociety.org/wp-content/uploads/sites/9/2021/08/2021_Data_Challenge.pdf
23. engrXiv preprint on C-MAPSS evaluation-protocol sensitivity: https://engrxiv.org/preprint/download/8145/13229/11468
24. Bi-cLSTM, residual-corrected bidirectional LSTM for RUL (includes chronological RMSE table): https://arxiv.org/pdf/2603.00745
25. “The significance of latent data divergence in predicting system degradation”: https://arxiv.org/pdf/2406.12914
26. MS_Transformer, *Acta Automatica Sinica* (C-MAPSS attributes and results): https://www.aas.net.cn/cn/article/doi/10.16383/j.aas.c230634
27. *Engineering Applications of Artificial Intelligence* 133:108475 (FD001/FD003 results): https://scholarworks.unist.ac.kr/handle/201301/83174
28. PerFormer, permutation-based vision transformer for RUL: https://arxiv.org/pdf/2506.00259

Foundational methods referred to from background knowledge (not re-verified in this session): Vovk, Gammerman & Shafer, *Algorithmic Learning in a Random World* (conformal prediction, Mondrian variants); Gibbs & Candès (2021), adaptive conformal inference under distribution shift; Bates et al. (2021) and Angelopoulos et al. (2021), risk-controlling prediction sets / Learn-then-Test; Clopper & Pearson (1934), binomial confidence limits; Bai, Kolter & Koltun (2018), temporal convolutional networks.